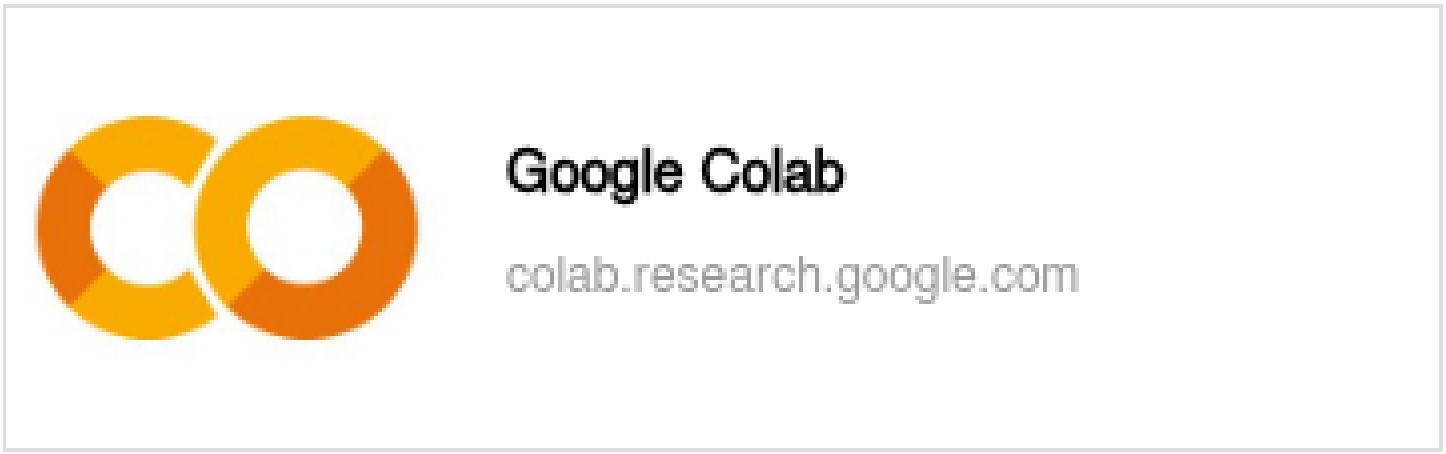
IBM - BUILDING RAG APPLICATIONS



GRADIO

Gradio is an open-source Python library used to quickly build customizable, interactive web interfaces for machine learning models, APIs, or Python functions.

Why Use Gradio?

Gradio is useful for several reasons:

- Ease of use: Gradio enables the creation of interfaces for models with just a few lines of code.
- Flexibility: Gradio supports various inputs and outputs, such as text, images, files, and more.
- Sharing and collaboration: Interfaces can be shared with others through unique URLs, facilitating easy collaboration and feedback collection

Construct a QA Bot that Leverages the LangChain and LLM to Answer Questions from Loaded Document

```python
from ibm_watsonx_ai.foundation_models import ModelInference
from ibm_watsonx_ai.metanames import GenTextParamsMetaNames as GenParams
from ibm_watsonx_ai.metanames import EmbedTextParamsMetaNames
from ibm_watsonx_ai import Credentials
from langchain_ibm import WatsonxLLM, WatsonxEmbeddings
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain_community.vectorstores import Chroma
from langchain_community.document_loaders import PyPDFLoader
from langchain.chains import RetrievalQA

import gradio as gr

# You can use this section to suppress warnings generated by your code:
def warn(*args, **kwargs):
    pass
import warnings
warnings.warn = warn
warnings.filterwarnings('ignore')

## LLM
def get_llm():
    model_id = 'mistralai/mistral-medium-2505'
    parameters = {
        GenParams.MAX_NEW_TOKENS: 256,
        GenParams.TEMPERATURE: 0.5,
    }
    project_id = "skills-network"
    watsonx_llm = WatsonxLLM(
        model_id=model_id,
        url="https://us-south.ml.cloud.ibm.com",
        project_id=project_id,
        params=parameters,
    )
    return watsonx_llm
```

```python
## Document loader
def document_loader(file):
    loader = PyPDFLoader(file.name)
    loaded_document = loader.load()
    return loaded_document

## Text splitter
def text_splitter(data):
    text_splitter = RecursiveCharacterTextSplitter(
        chunk_size=1000,
        chunk_overlap=50,
        length_function=len,
    )
    chunks = text_splitter.split_documents(data)
    return chunks

## Vector db
def vector_database(chunks):
    embedding_model = watsonx_embedding()
    vectordb = Chroma.from_documents(chunks, embedding_model)
    return vectordb

## Embedding model
def watsonx_embedding():
    embed_params = {
        EmbedTextParamsMetaNames.TRUNCATE_INPUT_TOKENS: 3,
        EmbedTextParamsMetaNames.RETURN_OPTIONS: {"input_text": True},
    }
    watsonx_embedding = WatsonxEmbeddings(
        model_id="ibm/slate-125m-english-rtrvr-v2",
        url="https://us-south.ml.cloud.ibm.com",
        project_id="skills-network",
        params=embed_params,
    )
    return watsonx_embedding
```

```python
## Retriever
def retriever(file):
    splits = document_loader(file)
    chunks = text_splitter(splits)
    vectordb = vector_database(chunks)
    retriever = vectordb.as_retriever()
    return retriever

## QA Chain
def retriever_qa(file, query):
    llm = get_llm()
    retriever_obj = retriever(file)
    qa = RetrievalQA.from_chain_type(llm=llm,
                                     chain_type="stuff",
                                     retriever=retriever_obj,
                                     return_source_documents=False)
    response = qa.invoke(query)
    return response['result']

# Create Gradio interface
rag_application = gr.Interface(
    fn=retriever_qa,
    allow_flagging="never",
    inputs=[
        gr.File(label="Upload PDF File", file_count="single", file_types=['.pdf'], type
        gr.Textbox(label="Input Query", lines=2, placeholder="Type your question here..
    ],
    outputs=gr.Textbox(label="Output"),
    title="RAG Chatbot",
    description="Upload a PDF document and ask any question. The chatbot will try to an
)

# Launch the app
rag_application.launch(server_name="0.0.0.0", server_port=7860, share=True)
```

Interface class core arguments

Gradio's Interface class wraps any arbitrary Python function. The interface class has the following three core arguments:

Term	Definition
Fn	Fn is the function to wrap. Each function parameter represents an input component, and the function's output should be either a single value or a tuple. Each element within the tuple corresponds directly to a specific output component.
inputs	The Gradio component(s) to use for the inputs of the function. The number of inputs should match the number of arguments in the function.
outputs	Outputs are the Gradio component(s) to use for the outputs of the function.

Common input types

Term	Definition
`checkbox`	A checkbox that can be set to `True` or `False` only.
`checkboxGroup`	An input type that allows users to select multiple values from a predefined checkbox list.
`dropdown`	An input type that provides a dropdown list where, by default, one value can be selected. If `multiselect` is set to `True`, then one or more values can be selected.
`file`	An input type that allows a user to upload a file.
`image`	An input type that allows the user to select or upload an image.
`radio`	An input type that forces the user to choose one value.
`slider`	An input type that provides a slider where a value must be selected between a minimum and a maximum range. The `value` parameter defines the default value, and `step` provides the increment value. Setting the minimum, maximum, and `step` values to integers will select integer values.
`textbox`	An expandible text box that allows the user to type in text.

Common output types

Term	Definition
`textbox`	An expandible text box that allows the user to type in text.
`label`	A type typically used for classification models. Can output the classes along with their predicted probabilities. You can control the number of output classes using the `num_top_classes` parameter.